Ss. Cyril and Methodius University in Skopje

Faculty of Computer Science & Engineering



Recovering Distributed LLM Teams After Information Changes

A paired experimental study of conversation retention and recovery

---

Student

Endrit Hasani

Mentors

Prof. Sonja Gievska, Ph.D.

Martina Toshevska, M.Sc.

Skopje, October 2026

Table of Contents

Page

1. Abstract 1

2. Introduction 2

2.1 The problem: useful work built on changing information 2
2.2 Research question and contribution 2
2.3 Why a second experiment was needed 2
2.4 Scope of the report 3

3. Related Work 4

3.1 What should be kept when information changes? 4
3.2 What memory-repair systems add 4
3.3 Why communication must be evaluated through outcomes 5
3.4 Reading information, revising answers, and acting on them 5
3.5 How interface and evaluation research shaped the refinement 5
3.6 From these lessons to the adopted experiment 6

4. System, Methods, and Experiment 7

4.1 A trial from start to finish 7
4.2 The numerical task and the source change 7
4.3 Information and assistance available to each agent 8
4.4 Messages, final answers, and turn limits 8
4.5 Keeping or clearing history fairly 9
4.6 Preparation and the decision to run the main experiment 9
4.7 Model and recorded execution 10
4.8 Measuring success, effort, and uncertainty 10

5. Results and Discussion 11

5.1 What the preparation showed 11
5.2 How often did the teams recover? 11
5.3 Legal communication was common; successful completion was not 12
5.4 What the interactions reveal 13
5.5 What explains the gap between communication and recovery? 15
5.6 How much work did recovery take? 15
5.7 What changed after the preliminary experiment? 17
5.8 Limitations 17
5.9 What should be tested next? 18

6. Conclusion 19

References 20

Appendix A. Complete main outcomes 22

Recovering Distributed LLM Teams After Information Changes

17 October 2026

1. Abstract

When information changes, a team of language-model agents may need to reconsider an answer it has already reached. Keeping the earlier conversation can preserve useful evidence, but it also leaves outdated conclusions in view. Clearing the conversation removes both. This study compares these two choices in a simple task: two agents hold separate private lists of integers and must both submit the largest value across the lists. After their first collaboration, the record containing the largest value is replaced with a lower value. The same team state is then continued twice, once with the earlier conversation and once without it.

A preliminary experiment revealed weak initial collaboration and frequent violations of the communication rules. A later literature review helped guide clearer instructions, better separation of current data from history, and a short coordination strategy. In the final assisted experiment, using Qwen3.5-9B, retaining history led to successful recovery in 2 of 20 trials (10%), while clearing it succeeded in 3 of 20 (15%). The difference was +5 percentage points in favor of clearing, with an exploratory 95% bootstrap interval of −15 to +25 points. This small study supports no general preference between the policies.

Initial collaboration and rule compliance were better than in the preliminary experiment, although different inputs and several design changes prevent a causal attribution to assistance. Recovery still failed often: 32 of the 35 failed recoveries used only legal actions. The interactions show why. Agents sometimes exchanged correct information without submitting answers, confused the sources of values, or stated the right answer without committing it. Better communication did not ensure that both agents finished correctly after information changed.

Keywords: large language models; multi-agent systems; information revision; recovery; conversation history; distributed coordination.

2. Introduction

2.1 The problem: useful work built on changing information

A team may finish a task and then learn that part of its information was wrong. A measurement may be corrected, a document updated, or an option withdrawn. The team must work out what needs to change. Some earlier work may still be useful, while other conclusions may no longer be valid.

This creates a practical choice for a team of large language model (LLM) agents. Here, an agent is a model participant with its own private information and conversation history. The two agents use the same model but receive separate inputs. Should they keep the earlier conversation, or start again with empty histories? Keeping everything avoids losing useful reports and calculations. It also leaves old answers available for reuse, even when they are no longer correct. Clearing everything removes those old answers, but makes the agents repeat work that might still be valid.

Consider two agents with separate lists of numbers. One reports 943 and the other reports 922, so both answer 943. The record containing 943 is then replaced with a smaller value. The report of 922 may still be correct because that agent's list has not changed. The other agent must inspect its updated list: knowing that its old maximum was 943 does not tell it the largest value that remains. To recover successfully, the team must combine new information with any earlier evidence that is still useful.

Ordinary text makes this harder to manage. A message may contain a source value, an interpretation, and a proposed answer in the same sentence. Another agent may reuse the answer without checking what supported it. Changing a source therefore does not automatically change the team's later decisions.

The numerical task in this study comes from the Global Maximum task family in SILO-BENCH, which evaluates LLM teams working with private information [1]. This study adds a source change and compares two ways of continuing afterward. It also defines its own rules for communication and final answers, so its success rates are not directly comparable to SILO-BENCH scores.

2.2 Research question and contribution

The study asks:

> *When a source change makes the previous answer wrong, how does keeping or clearing the earlier conversation affect a two-agent LLM team's ability to reach the new answer?*

The comparison takes place in one fixed environment with the same model, instructions, task, and call limits. In both conditions, agents receive their current private data and a notice of the change. The **retain** condition also gives them their earlier visible history. The **clear** condition removes it.

This is a comparison of two practical policies. History contains more than memory of an answer: it also contains evidence, mistakes, wording, and previous actions. Keeping it changes both the information available and the length of the input. The experiment measures these consequences together.

The contribution is a small, fully recorded study that connects final outcomes to what the agents actually did. It distinguishes four things that can easily be confused: following the action rules, communicating correct information, submitting an answer, and having both agents submit the correct answer. A conversation can look sensible while the team still fails at the last step.

2.3 Why a second experiment was needed

The preliminary experiment tested whether this recovery comparison could be run with conversations generated by the agents themselves. It used the same model checkpoint, the same two-agent maximum task, and the same kind of source change. Each history policy succeeded in 2 of 20 main trials. However, the original teams had solved only 3 of those 20 tasks before the change.

There was also a communication problem. Across the pilot and main interactions, 60 of 206 team actions broke the action rules even though their JSON syntax was valid. For example, an agent could write a useful message but combine it with an incompatible waiting action. The system then rejected the action and delivered nothing. Other traces showed numerical mistakes, confusion between record positions and values, and final answers that contradicted the explanation. With retained history, the withdrawn maximum appeared in recovery submissions in ten main trials.

A separate audit found no scoring, message-delivery, or branch-isolation defect that explained these results. The records were usable, but the research question needed a clearer setting: it was difficult to study recovery from collaboration when most teams had not collaborated successfully in the first place.

The next stage combined these observations with research on interfaces, memory, communication, and evaluation. The revised design explained the action rules more clearly, emphasized that both agents must submit, separated current data from history, and supplied a short strategy for comparing local maxima. It also added focused diagnostics, four complete pilot trials, fresh inputs, and a better-balanced execution schedule.

These changes required a new experiment. Earlier replies could not fairly stand in for responses to the new instructions. The final experiment therefore generated new interactions and kept all main trials, including ordinary failures. The two experiments show how the study developed, but they do not form a controlled test of assistance: the inputs and several design choices changed together.

2.4 Scope of the report

This report covers two agents, one numerical task, one announced source change, and two history policies. The small task makes the correct answer exact and the interactions easy to inspect. It allows a close study of recovery behavior, but offers limited evidence about larger teams, other models, or realistic research workflows. More selective ways of repairing memory were considered in the wider project and remain outside this experiment.

3. Related Work

3.1 What should be kept when information changes?

The problem begins with a distinction between an answer and the evidence for that answer. In the example above, the answer 943 is a conclusion about two lists. If its source record changes, that conclusion must be reconsidered. The other agent's report of 922 may still be useful. A recovery method should ideally tell these two pieces of earlier work apart.

This idea has a long history. Doyle's truth maintenance system records why a belief is held, so that changing an assumption can lead to reconsideration of the conclusions that depend on it [2]. The AGM theory gives a formal account of removing and revising beliefs [3]. These works helped frame the project around the support for a conclusion, rather than simply whether an old answer remains in memory. They do not imply that an LLM's explanation is a reliable record of that support.

There is a similar idea in ordinary software. DBSP updates query results as input data change [4]. *Build Systems à la Carte* separates the order of computation from the decision about which results need rebuilding [5]. In these systems, reuse works because relationships between inputs and outputs are defined. Free-form agent messages do not automatically provide such relationships. This difference helped motivate a clear boundary in our experiment: the current lists are source data; earlier reports are claims that the agents must interpret.

The experiment also needs a fair starting point for its two continuations. Chandy and Lamport show why messages, as well as local state, matter when recording the state of a distributed system [6]. Here, a central controller saves the interaction at the end of the original phase. Both recovery conditions start from that same recorded interaction. This uses a simple controlled boundary rather than proposing a new distributed snapshot method.

3.2 What memory-repair systems add

Recent agent research goes beyond keeping or deleting an entire history. *From Faulty Memories to Corrected Actions* tracks support for stored memories and uses it to guide rollback and replay after a fault is diagnosed [7]. MEMOREPAIR removes affected memory items before making validated replacements available [8]. REVISE follows dependencies between data and workflow steps to preserve work that remains valid after a revision [9]. The shared lesson is that selective recovery needs some account of which earlier results depend on the changed information.

These earlier influences helped define the limits of the present study. Our controller does not tell agents which sentences to trust, supply a graph of semantic dependencies, or repair their numerical claims. It provides current sources and applies a simple history policy. The agents must decide what the change means. This makes the study a test of behavior under two basic policies, rather than a new selective memory-repair system.

Two further studies help explain why having information is not enough. StateAuditor separates a memory update from a behavioral update: an agent can store corrected information while still responding in an outdated way [10]. Reclaim Evaluation examines the risk of retaining a conclusion while losing the source needed to recompute it [11]. Both make the distinction between stored content and successful use of that content important. Our setting differs from Reclaim because agents still receive current private lists and can request fresh reports. Its findings therefore help frame the question, but do not predict which policy should win here.

3.3 Why communication must be evaluated through outcomes

SILO-BENCH provides the task foundation and a central lesson: active communication is not the same as effective distributed computation [1]. In the maximum task, agents can send many messages without correctly combining their private information. This is why our analysis examines final answers as well as message counts.

Paired evaluation also makes mixed effects visible. The Causal Intervention Router study compares recovery with continued execution from the same state and reports cases that are helped as well as harmed [12]. An average difference alone can hide these opposing outcomes. We therefore report worlds where only retain succeeds, only clear succeeds, both succeed, or neither succeeds. There is an important difference in the setting: that study perturbs observations while leaving the underlying environment unchanged; ours changes an actual source record.

Independent–Communicate–Revise (ICR) fixes initial reasoning and then varies communication, including a no-message control [13]. It measures whether communication corrects mistakes and whether it preserves answers that were already right. Richer communication can increase both helpful and harmful influence. This supports looking closely at what messages achieve, rather than treating more discussion as an improvement. Unlike our agents, ICR agents initially receive complete problems.

These studies were reviewed after the preliminary experiment. They helped refine the evaluation and interpretation; they did not originate the project's earlier choice to use independent continuations.

3.4 Reading information, revising answers, and acting on them

A model that solves a task in one call may still struggle when the same task unfolds through several exchanges. Laban and colleagues study this gap between single-turn ability and multi-turn reliability [14]. That distinction matters for our full-information controls: finding a maximum when all numbers are visible at once is useful evidence of numerical ability, but it does not test coordination between two agents.

LongMemEval distinguishes finding information, reading it correctly, reasoning about time, and updating knowledge [15]. These are separate demands. If a number is plainly visible in an agent's current list but the agent reports another value, calling the error "forgetting" would miss what is observable. Belief-R makes a related distinction between changing an answer when evidence requires it and keeping an answer that remains valid [16]. Our source change always makes the old maximum wrong. It tests revision, but cannot test whether agents preserve correct answers after harmless changes.

Research on self-correction and debate gives another reason to be cautious about simple remedies. Huang and colleagues find that models cannot reliably correct reasoning merely by reconsidering it without external feedback [17]. ReConcile shows benefits from collaboration among diverse models using explanations, confidence, and aggregation [18]. These different findings depend on different procedures and comparison conditions. Neither tells us that extra calls or a vote would solve our task, where agents have private information and each must make an irreversible submission.

3.5 How interface and evaluation research shaped the refinement

The preliminary experiment showed that a well-formed reply can still fail before its content reaches the partner. JSONSchemaBench evaluates structured outputs using several measures, including compliance and output quality [19]. Tam and colleagues examine how format restrictions affect model performance [20]. Together, these works support separating format from substance. In our study, a reply can be valid JSON but violate the action rules. It can also follow every rule while reporting a false maximum.

Interface research helped guide the next design. SWE-agent shows that changing the interface around a model can change its behavior [21]. Results from τ^2-Bench show that workflow assistance has different effects across conditions [22]. FlowBench also distinguishes selecting an action from a supplied history from completing a live interaction [23]. These findings encouraged clearer instructions and complete

conversational pilots. They did not justify assuming that the revised interface would produce successful recovery.

CheckList's focused behavioral tests informed diagnostics for specific abilities, such as reading the current source, following an instruction, and making the required commitment [24]. MAST provides useful descriptions of repeated messages, failure to finish, ignored information, and differences between reasoning and action [25]. Who&When highlights how difficult it is to assign a failure to one agent or step [26]. We use these ideas to describe the traces carefully. The descriptions are not proof of an internal cause or a substitute for independent human annotation.

3.6 From these lessons to the adopted experiment

The design developed in stages. Earlier work on revision and dependencies framed the difference between a source and a conclusion. The preliminary traces then exposed practical problems with communication and task completion. Later reviews helped guide the assisted instructions, behavioral checks, and analysis of the final run. Keeping these stages separate avoids suggesting that every cited paper influenced the initial design.

Two evaluation principles also shaped the reporting. *AI Agents That Matter* argues that agent evaluation should account for cost and disclose development choices [27]. We therefore include effort spent on failed recoveries and explain how the shared original interaction is counted. τ-bench distinguishes a successful run from consistency across repeated trials [28]. Our twenty paired worlds use one saved generation schedule, so they provide a limited comparison rather than evidence of reliable repeated performance.

4. System, Methods, and Experiment

4.1 A trial from start to finish

Each trial, called a *world*, contains two private lists and a predetermined source change. An agent can read its own list but learns about the other through messages. The largest value in one list is its *local maximum*. Both agents must submit the *global maximum*: the largest value across both lists.

First, the agents collaborate on the original data. The system saves the conversation they actually produced, including mistakes and any submissions. It then applies the source change and runs two separate continuations. One keeps the earlier history; the other clears it. Both agents can act again in each continuation. A separate *full-information control* gives one model all forty values at once and asks for the maximum without communication.

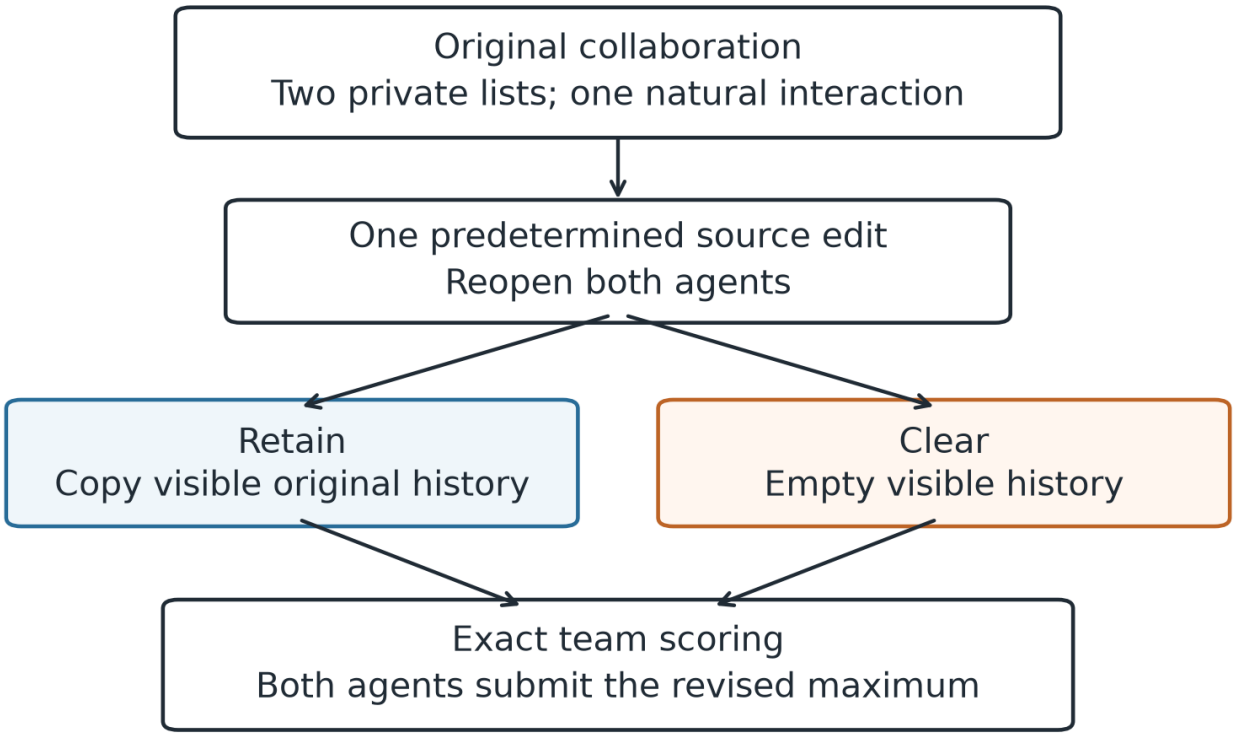


Figure 1. One paired trial. The original interaction runs once. Retain and clear then start independently after the same source change. Neither continuation can see messages or answers from the other. The full-information control tests maximum finding without the team interaction.

Pairing gives the policies the same original data, edit, and initial interaction. Their recovery inputs differ because one contains history. The study compares twenty main worlds, with separate worlds for preparation. Messages and calls are parts of a world, not additional independent trials.

4.2 The numerical task and the source change

Each agent holds twenty integers. A private part of a distributed dataset is often called a *shard*; here, a shard is simply one agent's list. Before the eligibility checks, each integer is sampled uniformly from −1000 through 1000. The lists are not sorted and the largest value is not marked.

Every accepted original input has one uniquely largest record. Let its value be M, and let M_2 be the largest value among the other thirty-nine records. We require M_2 to be greater than −1000. The edit replaces the record containing M with a randomly sampled integer below M_2:

$$x' \sim \text{Uniform}\{-1000, \ldots, M_2 - 1\}, \qquad M' = M_2 < M. \tag{1}$$

Thus the new correct answer is always M_2. It is lower than the old answer, so repeating the old maximum must fail. The owner of the changed record also cannot recover by reporting its old local maximum: it needs to inspect the values that remain.

Input generation also checks which agent owns the old winning record, so that this role can be balanced. Unsuitable proposals are rejected before any model is called. The final study uses four qualification worlds, four pilot worlds, and twenty main worlds. All complete original and revised inputs are distinct. They were also checked against locally available historical inputs, although this cannot rule out overlap with model training data.

The main set has five worlds for each combination of changed owner (0 or 1) and first speaker (0 or 1). The first speaker stays the same across phases. Inputs, edits, assignments, and seeds were saved before inference. No world was excluded or replaced because of its results.

4.3 Information and assistance available to each agent

On every call, an agent receives the common instructions, its identity, its call count, and whether its partner has finished. It also receives its visible history, any change notice, and its current private list. History appears before the current values, with explicit boundaries between earlier and current phases.

The assistance explains messaging, waiting, and submitting. It emphasizes that both agents must submit and that submissions are private and final for that phase. It suggests a simple strategy: establish both local maxima, compare them, and share useful information before finishing. The study tests whether agents carry out and adapt this supplied strategy, rather than discover the algorithm unaided. The controller does not force an exchange or check whether a numerical report is true.

After the edit, both agents are told which agent's list changed and at which record index. Only the owner sees the replacement value in its notice and current list. The instructions say that only the named record changes. An accurate earlier report from the unchanged list can therefore still be useful.

The agents do not receive the evaluator's answer, the partner's private list, the rule that the changed record was the unique winner, future edits, or the history-policy label. The old value is not presented again as current source data. Under retain, it can still appear in the earlier conversation.

4.4 Messages, final answers, and turn limits

A message and a final answer serve different purposes. A message tells the partner something. A submission records the sender's final answer and ends its participation in that phase. Writing "I will submit 922" in a message does not itself submit 922.

The model expresses its choice as a JSON object with three fields: `message`, `submit`, and `wait`. A message is text or null; a submission is an integer or null; waiting is true or false. Table 1 gives the allowed actions.

Choice	Effect
Send a message	Deliver its text to the partner
Submit an integer	Record a private final answer and finish the sender
Send and submit together	Deliver the text, then finish the sender
Wait alone	Block the sender until a message arrives
Invalid reply or illegal combination	Consume the call, deliver nothing, and record rule feedback

Table 1. Action choices and their effects. Following these rules does not imply that a message or answer is numerically correct.

Waiting must be chosen alone. Combining it with a message or submission is illegal, as is choosing no action without waiting. The system checks these public rules after generation. It does not correct false claims. Rejected text remains in the audit record, but the partner never receives it.

Each agent has at most three calls in each phase. Unused calls cannot be transferred to the other agent. Agents alternate when both can act. If one has finished or is waiting, the other may continue if eligible. A legal message is delivered before the next eligible call. Waiting ends only when a message arrives.

A submission cannot be changed within the phase. The partner can see that the sender has finished, but cannot see its answer unless the sender also messages it. The phase ends when both have submitted, when call opportunities run out, or when waiting prevents further progress. The system does not stop based on whether an answer is correct. If the phase ends with a missing submission, that is a known failure. Team success requires two correct submissions.

4.5 Keeping or clearing history fairly

Both recovery conditions reopen the agents after the edit. They reset call counters, waiting status, and submissions, and give each agent the same current private list and permitted change notice.

Retain copies each agent's actual visible history from the original interaction. This includes its legal actions and old submissions, received messages, and public feedback about rejected actions. Wrong claims are copied without correction. Raw rejected replies are kept only in the audit record; they are not turned into delivered messages.

Clear starts with empty visible histories. The common instructions, current lists, change notices, and fresh call limits are the same as under retain. Clearing therefore removes the conversation, but does not turn the trial into an ordinary first attempt: agents still receive a notice that information changed.

The two continuations are separate. A message or answer from one never enters the other. Per-agent random seeds are matched by call index, but the requests differ because their histories differ; equal seeds do not guarantee equal responses. Every call starts with fresh hidden model state. Here, "retaining history" means retaining visible text, not carrying a private reasoning process or model cache across calls.

This design compares whole histories. It cannot isolate the effect of one outdated sentence, or separate the influence of useful evidence from input length and wording.

4.6 Preparation and the decision to run the main experiment

Configuration, instructions, inputs, scoring, and requirements for main execution were fixed before the first call. Preparation checked state isolation and hardware feasibility. Eight qualification controls tested maximum finding. Four single-call diagnostics tested instruction following, current-source reading, and use of supplied history. Their outcomes had to be recorded, but no diagnostic accuracy threshold was required.

Four pilot trials then ran the original interaction, both recoveries, and full-information controls on original and revised inputs. These tested live conversation, beyond the supplied views used in diagnostics.

Main execution required complete pilot records, healthy execution, sufficient resources, at least 3/4 correct original teams, and at least 7/8 correct controls. Recovery success or a favorable policy difference was not required. Every main world then received the original interaction, both recoveries, and a revised control, including worlds with an incorrect original team.

Execution order was balanced: retain ran first in ten main worlds and clear in ten. Device assignment was also split ten-to-ten, with five worlds in each device/order combination. Both recoveries of a world ran on the same device. This avoided the preliminary schedule's problem, where device and policy order were perfectly linked.

4.7 Model and recorded execution

The study used a pinned Qwen3.5-9B checkpoint [29] on two NVIDIA Tesla T4 devices. The model used four-bit NF4 double quantization with FP16 computation. Generation settings were temperature 0.7, top-p 0.8, top-k 20, repetition penalty 1, presence penalty 0, and one beam. Limits were 8,192 input tokens and 512 output tokens. There was no fine-tuning, constrained decoding, or fallback to a separate thinking mode.

A Python controller recorded requests, raw replies, and resulting states in SQLite. Each device kept one model worker loaded for both agents of its assigned world. Replies were saved before their effects were applied, so completed calls could resume without resampling. Only permitted visible history passed into later calls.

All generations, including failures and diagnostics, counted toward a ceiling of 500 attempts. Wrong answers and illegal actions were not retried. No infrastructure retry occurred. Recorded free-memory headroom was at least about 6.78 GiB, above the required 1.5 GiB.

A separate offline reconstruction checked the saved inputs, requests, message histories, actions, final outcomes, and effort. It verifies that the records and calculations agree. It does not repeat GPU computation or provide independent human judgments about behavior.

4.8 Measuring success, effort, and uncertainty

The main outcome is exact team recovery. For world w, R_w is one if both agents submit the revised maximum under retain, and zero for a known failure. C_w is defined in the same way for clear. An unresolved or unexecuted episode would remain unknown rather than be counted as a failure. The estimated difference is

$$\hat{\delta} = \frac{1}{20} \sum_{w=1}^{20} (C_w - R_w). \qquad (2)$$

In words, we subtract retain's result from clear's result within each world, then average the twenty differences. A positive value favors clear. We also report how many worlds succeed under both policies, under only one, or under neither.

To show uncertainty, the prespecified analysis uses 10,000 paired bootstrap samples. Each draws five worlds with replacement from each changed-owner/first-speaker group. Both policy outcomes stay together whenever a world is drawn. The analysis recalculates the average difference and takes the central 95% of these differences using a fixed percentile rule. Exact seeds and calculation details are in the analysis code.

This is an exploratory interval. With only twenty worlds, it is not established that this procedure would include the underlying difference in 95% of repeated studies. Resampling saved worlds also cannot tell us how stable the model would be if the same worlds were executed again with new generations.

Effort is measured as generation time after the edit with the model already loaded, including failed recoveries. This *ready-model inference time* excludes loading and differs from elapsed notebook time and billing. The same paired bootstrap is used for the mean time difference. Calls, tokens, messages, completion, wrong agreement, and missing submissions are also reported. Complete-policy cost includes the original interaction, although it ran only once and was shared by both continuations.

5. Results and Discussion

5.1 What the preparation showed

All twenty intended main worlds were completed as experiments: their original interactions, both recoveries, and revised controls were recorded. No main outcome or effort value is unknown. The campaign used 356 physical first attempts and no retries or contingency attempts.

The eight qualification controls were all correct. The behavioral diagnostics were less reassuring. All four returned legal actions, but only one met its exact target. The others exposed weaknesses in instruction following, reading current values, and making the required commitment. This showed why numerical ability and rule compliance were insufficient readiness measures on their own.

The four live pilots produced four correct original teams and eight correct full-information controls. Yet all eight pilot recoveries failed. The main experiment still proceeded because the pilots met the fixed requirements: they demonstrated initial collaboration, numerical capability, complete records, and feasible execution. Recovery success was not an entry requirement. The candidate was not adjusted after these failures to obtain a favorable pilot result.

5.2 How often did the teams recover?

Retaining history succeeded in **2 of 20 worlds (10%)**. Clearing it succeeded in **3 of 20 (15%)**. The successes occurred in different worlds: two succeeded only with retain, three only with clear, and fifteen with neither. No world succeeded under both policies.

The difference is therefore (3 − 2)/20, or **+5 percentage points** in favor of clear. The exploratory 95% bootstrap interval is **−15 to +25 percentage points**. Only five worlds distinguish the policies on success, and one world changes a policy's success rate by five points. The result is too uncertain to establish which policy is better. It also does not show that their performance is equivalent.

Before the edit, ten of the twenty original teams were correct. Of those ten, two recovered under retain and one under clear. Clear's other two successes followed incorrect original teams. These small subset counts describe what happened; they are not a separate estimate of a policy effect. The primary comparison includes all twenty worlds because the study was designed to follow naturally generated collaboration, including mistakes.

The revised full-information controls were correct in 18 of 20 worlds. This shows that the model could often find the answer when given all the numbers at once. However, a control uses a different prompt, information arrangement, and seed. Its result is neither a guaranteed upper bound for the team nor a prediction of what that team should achieve. The gap points to the importance of interaction, but does not identify one cause of failure.

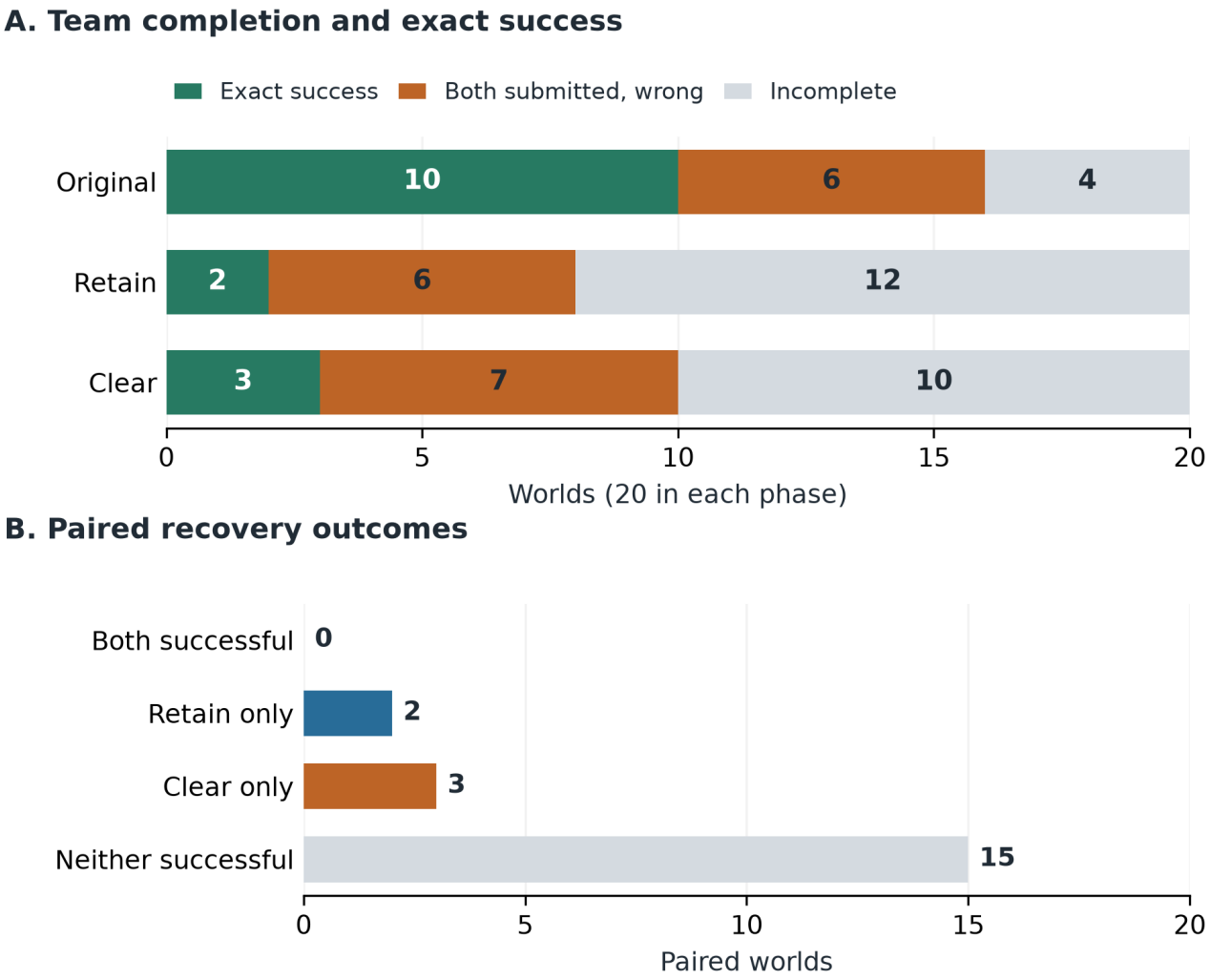


Figure 2. Main recovery outcomes. Panel A separates successful teams, teams that both submitted but were wrong, and teams with at least one missing submission. Each bar contains twenty worlds. Panel B shows which outcomes occurred together within the same world. An incomplete terminal episode is a known failure, not a missing observation.

5.3 Legal communication was common; successful completion was not

Eight of the campaign's 356 calls were hardware checks. The other 348 were scientific responses: 308 team actions and 40 controls or diagnostics. All 348 had valid JSON syntax and the required field types. Six were nevertheless illegal because they combined a message with waiting. All six occurred in main team interactions; the pilot team actions were all legal. Across pilots and main interactions, 302 of 308 team actions followed the rules.

Main-phase measure	Original	Retain	Clear
Calls	82	91	85
Illegal actions	1	1	4
Delivered messages	70	85	77
Both agents submitted, out of 20 teams	16	8	10
Exact success, out of 20 teams	10	2	3
Correct final answers, out of 40 agent slots	30	14	18
Wrong final answers, out of 40 agent slots	6	8	11
Known absent answers, out of 40 agent slots	4	18	11

Table 2. Communication and submissions in the main experiment. There are twenty teams and forty possible agent answers per phase. Call counts vary with behavior. Every absent answer is known to be absent; no slot has an unknown outcome.

Table 2 shows a large gap between talking and finishing. Retain delivered 85 messages, but only eight teams reached two submissions. Clear delivered 77 messages and ten teams reached two submissions. Retain's eighteen failures consisted of twelve incomplete teams and six completed but wrong teams. Clear's seventeen failures consisted of ten incomplete teams and seven completed but wrong teams.

Most failures did not involve rejected actions. **Seventeen of eighteen retain failures and fifteen of seventeen clear failures used only legal actions: 32 of 35 failed recoveries in total.** This does not tell us how many failures a different protocol would prevent. It does show that action rejection was not the main pattern in the recorded failures.

Repetition was also common. Counting exact duplicate messages from the same sender within an episode gives 31 duplicates under retain and 18 under clear. Thirteen agent sequences under retain repeated the same message on all three calls, compared with four under clear. These are literal text counts. A repeated message can sometimes help, and differently worded messages can still repeat an unproductive request. The examples below show why the particular repetition matters.

Neither policy submitted the withdrawn global maximum during final main recovery. Avoiding the old answer was therefore insufficient. History could still affect behavior in other ways, but these records do not establish how. Under clear, the old answer was absent from both current data and earlier conversation, so the two policies did not give equal opportunities to repeat it.

5.4 What the interactions reveal

The following examples connect the outcome counts to concrete behavior. World numbers refer to Appendix A. Worlds 00 and 01 were selected by the prespecified rule of taking the first clear-only and retain-only successes. The other cases were chosen afterward to illustrate different observed problems. They are not a complete or independently coded classification of causes.

Correct reports, but no answers

World 08 began with a correct original answer of 874. After the edit, the two local maxima were 758 and 857. The team therefore needed to answer 857. Under retain, both agents reported their current maxima correctly. Each then repeated the same message on its remaining calls.

Agent	Delivered message, repeated on all three calls	Final answer
0	"My local maximum is 758 at record 19. I will wait for your maximum before submitting."	Absent
1	"My local maximum is 857 at record 12. I will wait for your maximum before submitting."	Absent

Table 3. World 08 under retain. Both reports were correct and reached the partner before its next eligible call. Neither agent submitted an answer.

Both values needed for the comparison were in the conversation. The problem was no longer lack of access to those reports: neither agent turned them into a final answer. Although their messages said "wait," the actual `wait` fields were false. The agents were still sending messages, not blocked by the protocol. They used all six calls without submitting. No message was lost.

Clear failed differently on the same data. Agent 1 reported 786 even though its current list contained 857. Both agents then submitted 786. This was a finished, agreed, and wrong decision. The two continuations therefore illustrate different failures: correct information without completion, and completion based on an incorrect report.

Stating an answer is not submitting it

In world 04 under clear, agent 0 reported its correct maximum of 954 and later submitted it. Agent 1 correctly compared its own 839 with 954. Its last message said, "954 is the global max. I will submit." However, its `submit` field was null. Its partner had finished and it had no call left, so the team failed.

This case makes the scoring rule important. The agent's explanation contained the right answer, but its chosen action did not. Counting the sentence as a submission would change the recorded experiment after the fact. The distinction between reasoning and action, also emphasized in failure-analysis research [25], is necessary to explain the result.

Using a value from the wrong source

In world 09 under clear, agent 1 received a message reporting agent 0's maximum as 949 at record 16. The change notice separately identified agent 0's changed record as 13. Agent 1 then claimed that its partner's maximum was 695 at record 13 and submitted its own 836. The value 695 was actually at index 13 in **agent 1's own list**. The correct report of 949 was visible in the same request.

The record shows a mismatch between source ownership and record position: a value from the local list was described as belonging to the partner. It does not show how the model internally arrived at that claim. Because history was cleared, an outdated retained message is not needed to explain the presence of this error. The case also illustrates LongMemEval's distinction between information being available and being read correctly [15].

What successful recovery did and did not show

World 00 succeeded only under clear. The original answer was 943 and the revised answer was 922. Clear exchanged the correct current maxima, 900 and 922, and both agents submitted 922. Retain reported 837 for the changed list even though 900 was visible, repeated messages, and ended without submissions. The comparison shows different reading and completion behavior. It does not isolate one historical sentence as the cause.

World 01 succeeded only under retain. The unchanged agent's maximum of 926 appeared in the old history and was also reported again. The changed agent found its current maximum of 931, compared the two, and submitted 931; its partner followed. Clear exchanged the same correct maxima, but only one agent submitted. The other continued requesting information and finally waited. Retain's result is compatible with useful reuse of unchanged evidence. Since that evidence was also sent afresh, the record cannot show that old history was what made the difference.

A correct final answer can still rest partly on a false claim. Clear solved world 19 in two calls, with both agents submitting 896. Yet the first speaker described the partner's maximum as 639, a value in its own list. The partner's true maximum was 860. The local 896 exceeded both numbers, so the error did not change the answer. The team deserves credit for its exact result, while the intermediate claim gives reason to question how well it tracked sources.

Committing too soon and waiting too long

World 17 under retain shows the risk of an early commitment. An agent submitted 663 despite having 819 in its current list. Its submission was final, so later information could not change it.

World 11 under retain shows a different problem. One agent correctly submitted 950. The other then chose a legal wait, but its finished partner could no longer send a message to wake it. The phase ended in deadlock even though only one agent was waiting.

The agents needed both to gather enough evidence and to finish while they could. These traces describe failures at those two ends of the process. They do not establish whether extra calls or reversible submissions would have rescued the teams.

5.5 What explains the gap between communication and recovery?

Successful recovery required several steps to work together. An agent had to read its current list, keep track of whose values it was using, understand the change, use its partner's report, and finally submit. Correct messages helped with only part of this process. In the examples, agents sometimes had the required information but did not act on it. Elsewhere, a delivered report was replaced by a value from the wrong source.

This helps explain the central result without assuming an unseen internal cause. The final teams often followed the communication rules, yet failed to connect information to two correct final actions. Preserving or reacquiring useful content was not enough. The distinction between sources and conclusions in Reclaim remains relevant [11], but it is only part of what these teams needed to manage. Retained history might also encourage repetition of an ineffective exchange; that remains a hypothesis.

Recovery success was lower than original success: 10–15% rather than 50%. This decline cannot be attributed entirely to outdated history, since clear also performed poorly. Both recoveries introduce a change notice and reopen the task. Without a matched reopening in which no fact changes, the study cannot separate the effects of factual revision from general difficulty with reopening and coordination. Likewise, full-information controls alter several demands at once. Their higher accuracy cannot be converted into a percentage of failures caused by communication.

5.6 How much work did recovery take?

Measure, mean per main recovery	Retain	Clear
Model calls	4.55	4.25
Ready-model inference time	29.439 s	25.656 s
Input tokens	5,146.10	4,017.60
Generated tokens	170.55	159.70
Delivered messages	4.25	3.85

Table 4. Mean effort after the source change, across all twenty worlds. Failed recoveries are included. Time differences are calculated from unrounded records.

Clear used an average of **3.783 fewer ready-model seconds per world**. The exploratory 95% paired clear-minus-retain interval was **−8.241 to +0.589 seconds**, which includes zero. Its mean differences were −0.30 calls, −1,128.50 input tokens, −10.85 generated tokens, and −0.40 delivered messages per world. These observations do not establish a general efficiency advantage.

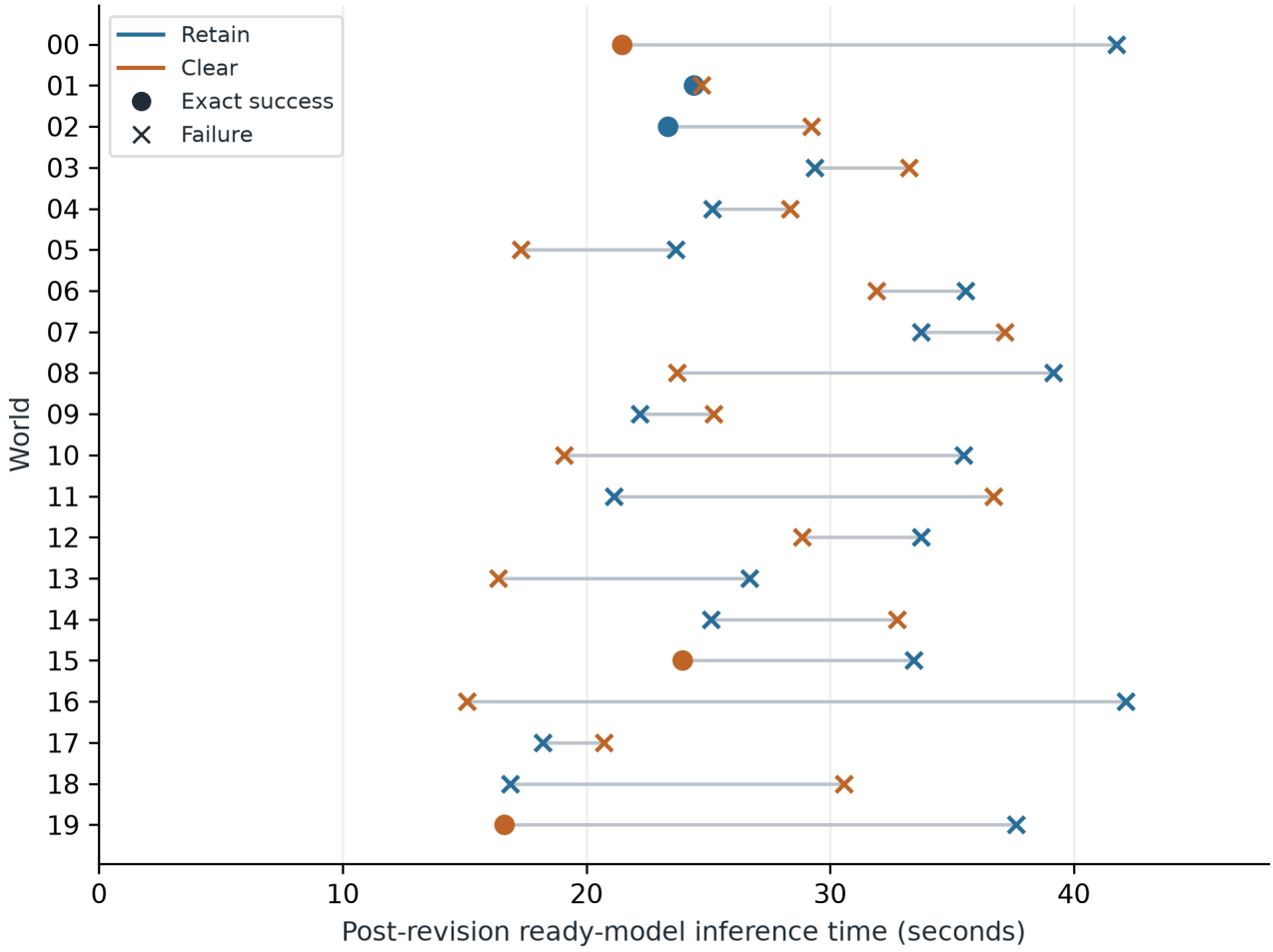


Figure 3. Time spent generating during each world's two recoveries, with the models already loaded. Lines connect the same world. Circles indicate exact success; crosses indicate known failure. Both agents' calls are included. A short recovery can be an early failure.

The last point matters when interpreting Table 4. In world 16, both policies ended with the same wrong pair of answers, [601,500]. Clear took about 15.11 seconds and retain 42.14 seconds. In world 11, retain used less time because it deadlocked early while clear continued. Less time did not necessarily mean that useful work was completed more efficiently.

The original interactions, shared by both policies, used 82 physical calls and 493.775 ready-model seconds. If each policy were run as a complete procedure from the beginning, retain would account for 173 calls and 1,082.553 seconds, and clear for 167 calls and 1,006.890 seconds. These are logical policy costs. Adding them would count the shared original interaction twice. Controls and hardware checks are research overhead and are not assigned to either policy.

The observed campaign lasted about 38.26 minutes. This is different from summed generation time: model loading and orchestration add time, while work on separate devices can overlap. Neither measure gives provider billing or energy use. All scientific replies ended normally, and the longest used 124 of the 512 permitted output tokens. The recorded failures were not caused by replies reaching that limit.

5.7 What changed after the preliminary experiment?

Observed measure	Preliminary experiment	Final assisted experiment
Original team success	3/20	10/20
Retain recovery success	2/20	2/20
Clear recovery success	2/20	3/20
Clear-minus-retain difference	0 points	+5 points
Exploratory success interval	[−10,+10] points	[−15,+25] points
Revised full-information controls correct	15/20	18/20
Illegal actions, all pilot/main team calls	60/206	6/308
Retain teams with both submissions	20/20	8/20
Clear teams with both submissions	19/20	10/20
Retain worlds submitting the obsolete answer	10/20	0/20

Table 5. Observations from two separate experiments. The inputs differ, and the same world number does not identify a matched trial across experiments. The preliminary interval uses its original analysis convention.

The final experiment had a 35-percentage-point higher original success rate, far fewer illegal actions, and no recovery submissions of the obsolete answer. In the main original interactions alone, illegal actions fell from 42 of 82 calls to 1 of 82. These are substantial observed differences. The comparison does not establish that assistance caused them.

Instructions, action examples, source placement, history boundaries, feedback, inputs, seeds, preparation checks, and execution assignments changed together. There is no comparison on the same worlds with assistance switched on and off. The final schedule also separates device from policy order, unlike the preliminary schedule. That improves the final design without turning the earlier experiment into a controlled comparison.

The change in behavior is informative even though it is not causal evidence. Preliminary retain often finished with an obsolete answer. Final retain more often continued sending messages without finishing. Its recovery calls increased from 44 to 91, while successful teams remained at two. Removing a conspicuous kind of mistake therefore did not produce reliable recovery. The two experiments should be discussed together as stages of the research, while their outcomes remain separate.

5.8 Limitations

A small sample without repeated executions. There are twenty worlds, but only five where the policies differ on success. Each world uses one saved generation schedule. The bootstrap resamples these worlds; it does not measure how often the same team would succeed if run again. Its small-sample coverage is also uncertain.

Several aspects of history change together. Retain preserves useful reports, obsolete claims, wording, and input length. The study cannot assign the result to one of these. Half the original teams were wrong, so the main finding concerns recovery after natural collaboration, including earlier failure. There is also no no-change reopening or harmless edit that leaves the answer unchanged. These would be needed to separate reopening from factual revision and to test preservation of still-valid answers [16].

A narrow task and model setting. The results concern one quantized model, two agents, a short call budget, and edits that always lower a unique maximum. Finding a maximum requires little computation. The task is useful for inspecting behavior, but says little about savings from reusing expensive work. Larger teams and professional workflows may present different difficulties.

Limits of behavioral interpretation. The trace review was AI-assisted and unblinded. It had no independent human coding, inter-rater agreement, or tests that changed one suspected cause and reran the case. The records show what was visible and what the agents did. They do not reveal private reasoning or identify the single correction that would rescue a failure [26].

Limits of runtime verification. Offline checks reconstruct the saved experiment. No crash or retry occurred, so the run does not empirically test those recovery paths. GPU numerical equivalence was not independently reproduced, and the final records cannot certify every hidden backend state.

5.9 What should be tested next?

The first priority is to test the whole path from reading to final action. Focused cases could check current-value extraction, source ownership, use of a delivered report, and submission after a partner finishes. Following the approach of CheckList [24], each test should target a clear behavior. Live interactions should follow, because using a supplied correct history differs from producing one with a partner.

A matched no-change reopening would help show whether the change itself creates extra difficulty. An edit that leaves the answer correct would test whether agents preserve valid conclusions. A narrower history comparison could keep useful evidence constant while varying whether the old conclusion is shown [11], [13]. These conditions ask different questions from keeping or clearing the whole conversation.

Fresh worlds and repeated continuations would provide stronger evidence about stability. Changes to the model, call limit, communication rules, or feedback should be tested as new conditions, with their costs included. They should not be used to replace or repair the outcomes already recorded.

6. Conclusion

This study asked whether keeping or clearing earlier conversation helps a two-agent LLM team recover after a source change. All twenty intended main pairs were recorded. Retain succeeded in 10% of worlds and clear in 15%. The difference of +5 percentage points had an exploratory interval from −15 to +25 points. The evidence does not support a general preference for either history policy.

The clearer lesson is that communication and completed decisions must be evaluated separately. Compared with the preliminary experiment, the final experiment had better initial collaboration and far fewer illegal actions. Different inputs and multiple design changes prevent attributing those improvements to assistance alone. Recovery nevertheless remained poor. Agents sometimes exchanged the right values but never submitted, used a value from the wrong source, or described the correct answer without choosing it as an action. Even a successful final decision could contain a false intermediate claim.

Keeping evidence is useful only if agents can recognize what remains valid and act on it. Clearing history avoids some old content but does not remove the need to read, coordinate, and finish. In this setting, improved initial collaboration and compliant communication did not ensure that both agents reached the correct decision after information changed.

Code and data availability. The private companion repository, *llm-team-recovery-study*, contains the executed notebooks, unchanged Kaggle downloads, experiment source, configuration, and offline analysis for both experiments. Access must be granted to readers. Exact inputs, prompts, seeds, runtime versions, and trace locations are kept there. One supporting prose ledger has a large-integer transcription error: 354 of its 356 printed per-call seed labels differ from the saved integers. Reconstruction uses the exact saved request values. Experimental outcomes and reported statistics are unaffected; the original records have not been changed.

Assistance disclosure. AI tools assisted drafting, editorial revision, and qualitative trace review. The numerical checks do not replace an independent human review of the behavioral interpretations.

References

[1] Y. Zhang, F. Liu, Y. Shan, X. Huang, X. Yang, Y. Zhu, X. Cheng, C. Liu, K. Zeng, T. J. Zhang, and W. Jiang, "SILO-BENCH: A Scalable Environment for Evaluating Distributed Coordination in Multi-Agent LLM Systems," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics, Volume 1: Long Papers*, 2026, pp. 29379–29398. DOI: 10.18653/v1/2026.acl-long.1354.

[2] J. Doyle, *A Truth Maintenance System*. MIT Artificial Intelligence Laboratory, Memo AIM-521, June 1979. Institutional publication record.

[3] C. E. Alchourrón, P. Gärdenfors, and D. Makinson, "On the Logic of Theory Change: Partial Meet Contraction and Revision Functions," *The Journal of Symbolic Logic*, vol. 50, no. 2, pp. 510–530, 1985. DOI: 10.2307/2274239.

[4] M. Budiu, T. Chajed, F. McSherry, L. Ryzhyk, and V. Tannen, "DBSP: Automatic Incremental View Maintenance for Rich Query Languages," *Proceedings of the VLDB Endowment*, vol. 16, no. 7, pp. 1601–1614, 2023. DOI: 10.14778/3587136.3587137.

[5] A. Mokhov, N. Mitchell, and S. Peyton Jones, "Build Systems à la Carte," *Proceedings of the ACM on Programming Languages*, vol. 2, ICFP, article 79, pp. 1–29, 2018. DOI: 10.1145/3236774.

[6] K. M. Chandy and L. Lamport, "Distributed Snapshots: Determining Global States of Distributed Systems," *ACM Transactions on Computer Systems*, vol. 3, no. 1, pp. 63–75, 1985. DOI: 10.1145/214451.214456.

[7] C. Yu, Y. Wang, J. Zhang, Y. Duan, M. Zheng, Z. Wu, K. Shi, and T. Cai, "From Faulty Memories to Corrected Actions: Dependency-Guided Rollback Repair for Memory-Augmented Agents," arXiv:2608.10502v1, 2026, preprint. Versioned publication.

[8] Y. Zhao, C. Dai, M. Kou, and Y. Xiu, "MEMOREPAIR: Barrier-First Cascade Repair in Agentic Memory," arXiv:2605.07242v1, 2026, preprint. Versioned publication.

[9] R. Qi, X. Wu, P. Liu, J. Chen, and Y. Liu, "REVISE: Validity-Guided Recovery for Online Revisions in Agent Workflows," arXiv:2609.00643v1, 2026, preprint. Versioned publication.

[10] H. Sun and L. He, "When Memory Updates but Behavior Does Not: Repairing Implicit Stale Dependencies in Personalized Agent Responses," arXiv:2608.01619v1, 2026, preprint. Versioned publication.

[11] A. Kwon, "Reclaim Evaluation: A Lossy Memory Is Worse Than an Empty One," arXiv:2606.25449v5, 2026, preprint. Versioned publication.

[12] S. Xiao, S. Wang, X. Chen, K. Chao, M. Cui, F. Qian, C. Mei, F. Meng, Z. Yu, and J. Yin, "When Harnesses Lose the Signal: Causal Evaluation of Recovery in LLM Agents," arXiv:2610.00372v1, 2026, preprint. Versioned publication.

[13] S. Li, W. Yang, P. Zhang, A. Cheng, H. Ping, and P. Bogdan, "Beyond Final Accuracy: Auditing Communication in LLM Multi-Agent Systems," arXiv:2610.01042v1, 2026, preprint. Versioned publication.

[14] P. Laban, H. Hayashi, Y. Zhou, and J. Neville, "LLMs Get Lost In Multi-Turn Conversation," in *International Conference on Learning Representations*, 2026. Reading version: arXiv:2505.06120v1, 2025; identity with the final conference text is not assumed. Institutional publication record.

[15] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory," in *International Conference on Learning Representations*, 2025. Reading version: arXiv:2410.10813v2.

[16] B. Wilie, S. Cahyawijaya, E. Ishii, J. He, and P. Fung, "Belief Revision: The Adaptability of Large Language Models Reasoning," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 10480–10496. DOI: 10.18653/v1/2024.emnlp-main.586.

[17] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large Language Models Cannot Self-Correct Reasoning Yet," in *International Conference on Learning Representations*, 2024. Reading version: arXiv:2310.01798v2.

[18] J. Chen, S. Saha, and M. Bansal, "ReConcile: Round-Table Conference Improves Reasoning via Consensus among Diverse LLMs," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, Volume 1: Long Papers*, 2024, pp. 7066–7085. DOI: 10.18653/v1/2024.acl-long.381.

[19] S. Geng, H. Cooper, M. Moskal, S. Jenkins, J. Berman, N. Ranchin, R. West, E. Horvitz, and H. Nori, "JSONSchemaBench: A Rigorous Benchmark of Structured Outputs for Language Models," arXiv:2501.10868v3, 2025, preprint. Versioned publication.

[20] Z. R. Tam, C.-K. Wu, Y.-L. Tsai, C.-Y. Lin, H.-y. Lee, and Y.-N. Chen, "Let Me Speak Freely? A Study On The Impact Of Format Restrictions On Large Language Model Performance," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 2024, pp. 1218–1236. DOI: 10.18653/v1/2024.emnlp-industry.91.

[21] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. DOI: 10.52202/079017-1601.

[22] V. Barres, H. Dong, S. Ray, X. Si, and K. R. Narasimhan, "τ²-Bench: Evaluating Conversational Agents in a Dual-Control Environment," in *Proceedings of the 43rd International Conference on Machine Learning*, PMLR, vol. 306, 2026, pp. 6736–6782. Proceedings record. Accessible method text checked in arXiv:2506.07982v1, 2025.

[23] R. Xiao, W. Ma, K. Wang, Y. Wu, J. Zhao, H. Wang, F. Huang, and Y. Li, "FlowBench: Revisiting and Benchmarking Workflow-Guided Planning for LLM-based Agents," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 10883–10900. DOI: 10.18653/v1/2024.findings-emnlp.638.

[24] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond Accuracy: Behavioral Testing of NLP Models with CheckList," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 4902–4912. DOI: 10.18653/v1/2020.acl-main.442.

[25] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. A. Zaharia, J. E. Gonzalez, and I. Stoica, "Why Do Multi-Agent LLM Systems Fail?" in *Advances in Neural Information Processing Systems*, vol. 38, Datasets and Benchmarks Track, 2025. DOI: 10.52202/085713-4082.

[26] S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen, and Q. Wu, "Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems," in *Proceedings of the 42nd International Conference on Machine Learning*, PMLR, vol. 267, 2025, pp. 76583–76599. Proceedings record.

[27] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, and A. Narayanan, "AI Agents That Matter," arXiv:2407.01502v1, 2024, preprint version consulted. Versioned publication.

[28] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains," in *International Conference on Learning Representations*, 2025. Conference paper.

[29] Qwen Team, *Qwen3.5-9B: Model Repository and Model Card*, 2026, checkpoint used in this study. Pinned model repository.

Appendix A. Complete main outcomes

Answers appear in the order [agent 0, agent 1]. A dash means the agent had not submitted when the phase ended. **B** means both agents submitted, **H** means the call limit was reached, and **W** means no further action was possible because of waiting. These labels explain how a phase ended, not whether its answers were correct. World numbers refer only to the final assisted experiment.

World	Old → revised maximum	Original answers	Retain answers / end	Clear answers / end
00	943 → 922	[943,943]	[—,—] H	[922,922] B
01	956 → 931	[956,956]	[931,931] B	[—,931] H
02	976 → 964	[976,976]	[964,964] B	[964,—] H
03	946 → 893	[—,946]	[810,893] B	[876,—] H
04	972 → 954	[972,972]	[—,839] H	[954,—] H
05	952 → 884	[952,868]	[884,—] H	[884,868] B
06	941 → 917	[941,941]	[917,—] H	[766,—] H
07	991 → 980	[991,991]	[—,—] H	[—,980] H
08	874 → 857	[874,874]	[—,—] H	[786,786] B
09	979 → 949	[979,836]	[949,836] B	[949,836] B
10	991 → 987	[878,991]	[—,987] H	[878,987] B
11	984 → 950	[—,984]	[—,950] W	[950,—] H
12	963 → 905	[963,—]	[—,—] H	[—,—] H
13	992 → 873	[992,992]	[873,864] B	[873,864] B
14	980 → 975	[980,980]	[975,—] H	[975,—] H
15	917 → 815	[815,917]	[—,—] H	[815,815] B
16	702 → 601	[702,—]	[601,500] B	[601,500] B
17	974 → 955	[974,955]	[663,955] B	[—,955] H
18	961 → 912	[961,961]	[805,805] B	[764,805] B
19	931 → 896	[860,931]	[—,—] H	[896,896] B

Table A1. Every intended main world. No absent answer or unfavorable outcome is excluded. Revised full-information controls matched the revised maximum in every row except world 15 (881 instead of 815) and world 16 (585 instead of 601).